

WEB DEVELOPMENT

CSS & Responsive Design

Cascading Style Sheets · Layouts · Media Queries · Responsive Principles

Prof. Joshua Pacer | Web Development Course

Academic Year 2025–2026 | Module 4 - 5



Learning Objectives

By the end of this module, students will be able to:

- 01 Understand CSS syntax, selectors, and property declarations
- 02 Apply the CSS Box Model to control layout, spacing, and borders
- 03 Build flexible layouts using Flexbox and CSS Grid
- 04 Apply responsive design principles for multi-device compatibility
- 05 Write effective media queries and use responsive CSS units



What is CSS?

Cascading Style Sheets (CSS) is a stylesheet language used to describe the presentation of HTML documents. It controls how elements look, are arranged, and respond across different devices.

```
h1 { color: blue; }
```

Structure

HTML defines the content and document structure

Style

CSS handles visual presentation: colors, fonts, spacing

Behavior

JavaScript adds interactivity and dynamic content

CSS was developed by the W3C (World Wide Web Consortium) and first introduced in 1996. Today's standard is CSS3, which supports animations, transitions, and responsive features.

A CSS rule consists of a selector and a declaration block containing one or more property-value pairs:

```
/* Selector + Declaration Block */
```

```
h1 {  
  color: navy;  
  font-size: 24px;  
  margin: 0 auto;  
}
```

Selector

Targets the HTML element(s) to style (e.g., h1, .class, #id)

Property

The CSS attribute to change (e.g., color, font-size, margin)

Value

The setting for the property (e.g., navy, 24px, 0 auto)

Declaration

A complete property: value; pair inside the block

💡 *Best Practice: CSS can be written inline, in a <style> tag, or in an external .css file. External stylesheets (linked via <link>) are preferred for maintainability and reuse.*

💡 *Best Practice: External stylesheets (linked via <link>) are the preferred approach over inline styles or <style> tags. They separate concerns, promote code reuse, and make maintenance far easier across multi-page websites.*

CSS Selectors

Slide 5 of 17

Selectors define which HTML elements receive the styles. CSS has several types of selectors with different specificity levels:

Element

`p, h1, div`

Targets all elements of that type

Class

`.card, .btn`

Targets elements with that class attribute

#ID

`#header, #nav`

Targets a single unique element by id

Universal

`*`

Targets ALL elements on the page

Attribute

`[type='text']`

Targets elements with specific attributes

Pseudo

`:hover, :nth-child`

Targets elements in specific states

 *Specificity Order: ID (100) > Class (10) > Element (1) > Universal (0). Higher specificity overrides lower.*



CSS Properties Overview

CSS properties are categorized into logical groups. Understanding categories helps you quickly find and apply the right property:

Typography

font-family

font-size

font-weight

line-height

text-align

letter-spacing

Color & BG

color

background-color

background-image

opacity

background-size

gradient

Box Model

width

height

padding

margin

border

box-sizing

Display & Pos

display

position

top/left

z-index

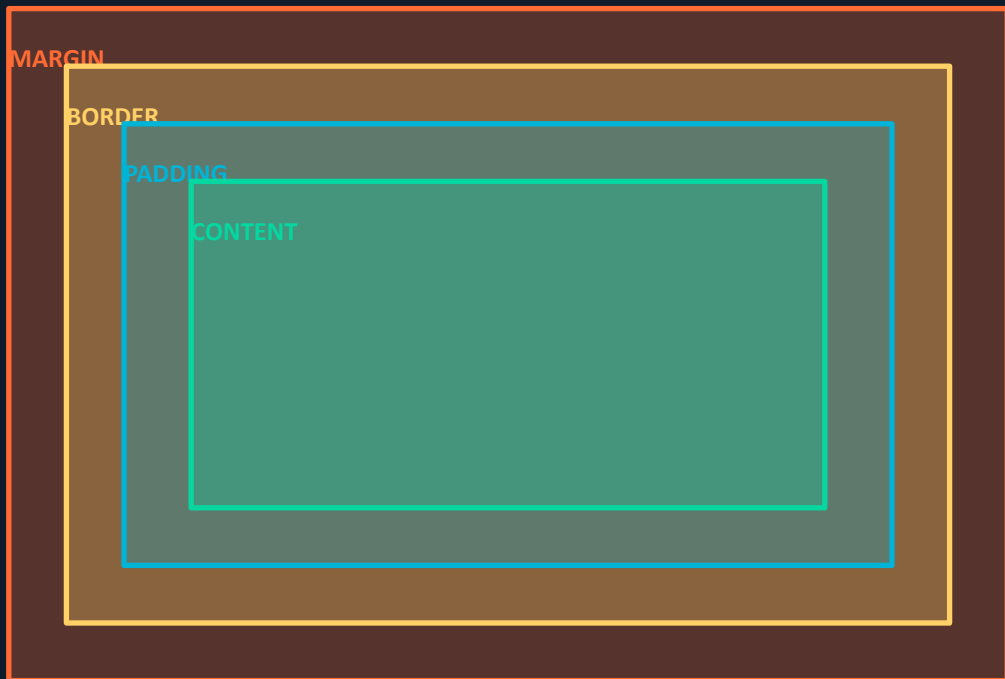
float

overflow



The CSS Box Model

Every HTML element is rendered as a rectangular box. The CSS Box Model describes how these boxes are sized and spaced using four key layers:



Content

The actual content area (text, images). Set with width and height.

Padding

Space between content and border. Transparent, inherits background.

Border

Surrounds padding and content. Can be styled with width, style, color.

Margin

Space outside the border. Used to separate elements. Transparent.



Box Model: Sizing & box-sizing

content-box (default)

```
.box {  
  box-sizing: content-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}  
/* Total = 250px */
```

border-box (recommended)

```
.box {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}  
/* Total = 200px */
```

content-box adds padding and border ON TOP of the specified width, making elements larger than expected. border-box INCLUDES padding and border within the width, making it far more predictable for layout design.

```
/* Global best practice */  
*, *::before, *::after { box-sizing: border-box; }
```




Flexbox — Flexible Box Layout

Flexbox is a one-dimensional layout model that distributes space along a single axis. It makes alignment, spacing, and order effortless without floats or positioning hacks.

```
display: flex; justify-content: space-between;
```

Item 1

Item 2

Item 3

Item 4

Flexbox is ideal for navigation bars, toolbars, card rows, and centering elements both horizontally and vertically with minimal code.

`display: flex`

Activates flex layout on the parent

`flex-direction`

row | column | row-reverse | column-reverse

`justify-content`

flex-start | center | space-between | space-around

`align-items`

stretch | center | flex-start | flex-end | baseline

`flex-wrap`

nowrap | wrap | wrap-reverse

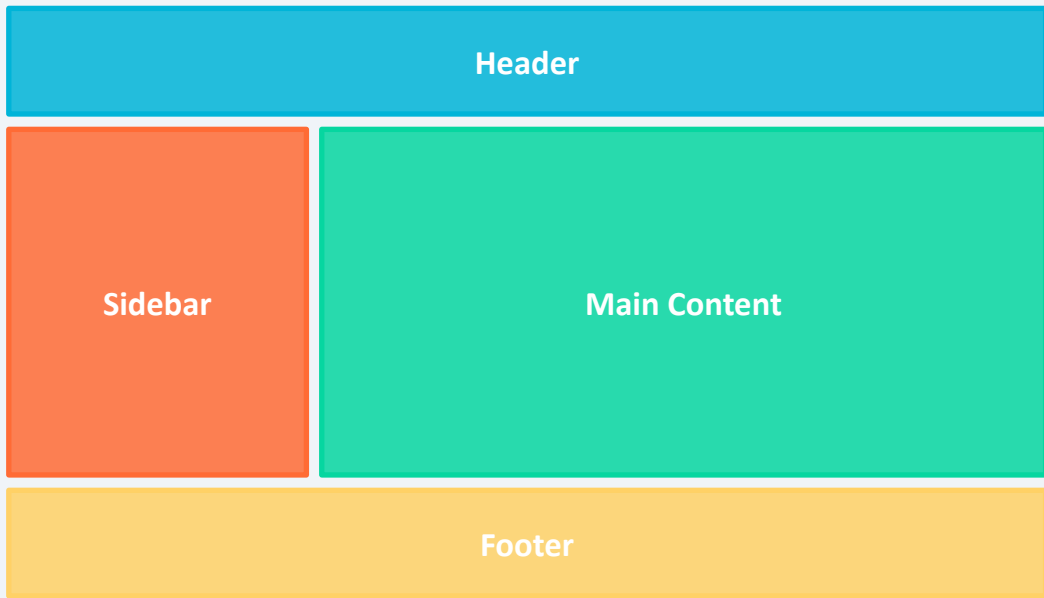
`flex-grow / shrink`

Controls how items grow or shrink in available space



CSS Grid Layout

CSS Grid is a two-dimensional layout system that divides a container into rows AND columns simultaneously. It's the most powerful CSS layout tool available for page-level design.



```
.container {  
  display: grid;  
  grid-template-columns:  
    200px 1fr;  
  grid-template-rows:  
    80px 1fr 60px;  
  gap: 16px;  
  height: 100vh;  
}
```

Key Difference: Flexbox = 1D (row OR column). Grid = 2D (rows AND columns). Use Grid for page layout; Flexbox for component-level alignment.



Flexbox vs. CSS Grid

Slide 11 of 17

Feature	Flexbox	CSS Grid
Dimensions	1D (row or column)	2D (rows and columns)
Best Use Case	Component alignment	Full page layout
Content Flow	Content-driven sizing	Layout-driven sizing
Browser Support	IE11+ (partial)	IE11 partial, modern full
Gap Support	gap (modern)	gap (first-class)
Item Placement	Order-based	Precise row/col placement



Pro Tip: Use Grid and Flexbox together! Grid for the macro page layout, Flexbox for micro-level component alignment. They complement each other perfectly.



Responsive Design Principles

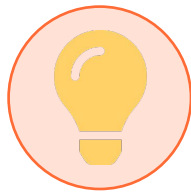
Responsive Web Design (RWD), coined by Ethan Marcotte (2010), ensures websites work beautifully across all screen sizes using three core techniques:

Fluid Grids



Use percentage-based widths instead of fixed pixel values. Elements scale proportionally to the viewport rather than remaining fixed.

Flexible Images



Images scale within their containing elements using `max-width: 100%`. This prevents images from overflowing their containers on smaller screens.

Media Queries



CSS rules that apply only when specific viewport conditions are met (screen width, device orientation, resolution), enabling device-targeted styles.



Did you know? Over 55% of global web traffic comes from mobile devices. Responsive design is no longer optional — it's a fundamental professional requirement.



Mobile-First Design Strategy

Mobile-First means writing your base CSS for the smallest screen, then using min-width media queries to progressively enhance for larger screens. This is the industry standard approach.

Mobile



320px-480px

Single column, large tap targets, minimal imagery

Tablet



481px-1024px

2-column layout, larger images, more content visible

Desktop



1025px+

Full multi-column layout, hover effects, rich content

```
/* Mobile base */ ... @media (min-width: 768px) { /* tablet */ } @media (min-width: 1200px) { /* desktop */ }
```

Q Media Queries

Media queries allow CSS to apply different styles based on the characteristics of the user's device. They are the backbone of responsive design:

```
@media screen and (min-width: 768px) {  
  .container {  
    display: grid;  
    grid-template-columns: 1fr 1fr;  
  }  
}
```

Common Breakpoints: 320px (mobile sm), 480px (mobile lg), 768px (tablet), 1024px (laptop), 1200px (desktop), 1440px (large desktop)

Media Types

- › screen – Computer/tablet/phone
- › print – For printed pages
- › all – Applies to all devices

Width Features

- › min-width: apply above N
- › max-width: apply below N
- › width: exact width match

Other Features

- › orientation: portrait/landscape
- › resolution: screen density
- › prefers-color-scheme: dark



Responsive CSS Units

Slide 15 of 17

Choosing the right unit makes the difference between a rigid and a truly responsive design. Here are the key CSS units and when to use each:

px

Absolute

Fixed pixels. Use for borders, shadows. Avoid for font-size, layout widths.

`border: 1px solid`

%

Relative

Relative to parent element. Great for fluid widths and heights.

`width: 50%`

em

Relative

Relative to the font-size of the parent element. Good for spacing.

`padding: 1.5em`

rem

Root-relative

Relative to the root (<html>) font-size. Best for font sizes.

`font-size: 1.2rem`

vw/vh

Viewport

Relative to viewport width/height (1vw = 1% of viewport width).

`width: 80vw`

fr

Grid-only

Fractional unit for CSS Grid. Divides available space proportionally.

`1fr 2fr 1fr`



Putting It All Together

A full responsive layout example combining Grid, Flexbox, Media Queries, and responsive units:

```
*, *::before, *::after { box-sizing: border-box; }

.page-layout {
  display: grid; grid-template-columns: 1fr;
  gap: 1rem; padding: 1rem;
}

.nav { display: flex; justify-content: space-between; align-items: center; }

@media (min-width: 768px) {
  .page-layout
  { grid-template-columns: 240px 1fr; }
}

@media (min-width: 1200px) {
  .page-layout
  { max-width: 1200px; margin: 0 auto; }
}
```

This pattern applies a global box-sizing reset, sets a single-column mobile layout, uses Flexbox for the navigation, and progressively enhances to a two-column sidebar layout on tablets and caps width on large desktops.



Summary & References

Slide 17 of 17

Key Takeaways



CSS uses selectors + property:value declarations to style HTML



The Box Model (content → padding → border → margin) governs all layout



Flexbox handles 1D layouts; CSS Grid handles 2D page structures



Responsive design requires fluid grids, flexible images, and media queries



Prefer rem/em/vw/% units over px for responsive, accessible designs

References

MDN Web Docs

developer.mozilla.org/en-US/docs/Web/CSS

W3Schools CSS

w3schools.com/css

CSS-Tricks

css-tricks.com

A Complete Guide to Flexbox

css-tricks.com/snippets/css/a-guide-to-flexbox

A Complete Guide to Grid

css-tricks.com/snippets/css/complete-guide-grid

Responsive Web Design (Marcotte)

alistapart.com/article/responsive-web-design

Google Web Fundamentals

web.dev/responsive-web-design-basics